

基于 MPI 的分布式 ANN 检索并行化

数据分片、混合并行、通信策略与图索引组合分析

姓名 匡航远 学号 2412235
专业 计算机科学与技术 日期 2026 年 5 月 31 日
GitHub <https://github.com/xzxxntxdy/Parallel-programming-NKU>

摘要

本文实现并分析了 DEEP100K 向量检索任务的 MPI 并行 ANN 查询框架。实现采用 base-sharding: rank 0 读取 base/query/ground truth, 使用 `MPI_Scatterv` 将 base 向量按连续区间分发到各 MPI 进程, 使用 `MPI_Bcast` 广播 query; 每个 rank 在本地分片上构建 IVF 或 HNSW 索引, 维护局部 top-100 候选, 最后由 rank 0 合并为全局 top-100 并计算 **recall@100**。在统一框架下实现了 Flat、IVF、HNSW、multi-HNSW 和 IVF-HNSW, 并比较 blocking、nonblocking、point-to-point、one-sided、`MPI_THREAD_MULTIPLE`、MPI+OpenMP 混合并行以及 scalar/SIMD 距离核。

x86 正式评测采用 100k base、1000 query, 共 172 组结果; ARM/Kunpeng 先采用 quick 规模 50k base、300 query 完成 111 组筛选, 再对 HNSW 最优候选执行 100k base、1000 query 的 PBS/qsub 全量验证。由于 ARM quick 只使用一半 base, 但仍对照 100k ground truth, Flat 精确扫描的 raw recall@100 上限约为 0.5027, 因此 ARM quick 的 recall 用于解释 reduced-base 趋势, 不参与最终 recall@100 ≥ 0.95 阈值筛选。x86 full 中, 在 recall@100 ≥ 0.95 下整体最低延迟为 HNSW 的 0.0312 ms/query; 若限定 MPI ranks ≥ 2 , 2 ranks + 4 OpenMP threads 的 HNSW 达到 0.0330 ms/query、recall@100=0.9816。ARM full qsub 中, 同一 HNSW P2T4 配置达到 0.1318 ms/query、recall@100=0.9814, 验证了 ARM 平台上的最终阈值结果。阶段计时表明, MPI 分片显著降低本地搜索时间, 但进程数增大后通信和全局 merge 的比例上升; 跨平台性能差异不仅来自 SIMD 指令宽度, 也来自核心频率、内存层次、MPI 运行时、作业调度环境和不同规模下的阶段比例。

目录

1 问题定义与评价指标	3
1.1 ANN 与 recall@100	3
1.2 quick 规模的解释	3
2 MPI 任务分配与全局 top-k 合并	3
2.1 base-sharding 数据划分	3
2.2 局部候选与全局 merge	4

3	本地索引与混合并行实现	6
3.1	Flat 与 SIMD 距离核	6
3.2	IVF 本地倒排索引	6
3.3	HNSW、multi-HNSW 与 IVF-HNSW	7
3.4	OpenMP 与 std::thread	7
3.5	MPI 通信模式	8
4	实验设置	9
4.1	平台与数据	9
4.2	实验矩阵	9
5	总体结果与跨平台趋势	10
6	IVF 参数消融与扩展性	13
7	通信方法、同步与 MPI 线程支持	14
7.1	MPI_THREAD_MULTIPLE	15
8	阶段 profiling、负载均衡与 cache locality	15
9	OpenMP 混合并行与 SIMD	16
10	图索引组合与负优化分析	18
11	综合分析	19
11.1	串行、MPI、MPI+SIMD、MPI+OpenMP 的关系	19
11.2	通信与计算重叠	19
11.3	merge/reduce 优化	19
11.4	局部 top-p 与 rerank	19
11.5	自由探索：候选返回与分层合并	20
11.6	平台差异	20
11.7	优化过程复盘	20
11.8	最终选择	21
12	结论	21
A	生成式 AI 辅助学习记录	22

1 问题定义与评价指标

1.1 ANN 与 recall@100

给定向量集合 $X = \{x_1, \dots, x_N\} \subset \mathbb{R}^d$ 和查询向量 q ，精确 k 近邻检索返回距离最小的 k 个向量。DEEP100K 使用内积相似度，本文沿用已有 SIMD 阶段的距离定义：

$$\delta(q, x) = 1 - \sum_{i=1}^d q_i x_i. \quad (1)$$

距离越小表示向量越接近。ANN 的目标是在高召回下减少访问向量数和单 query 延迟。本文固定 $k = 100$ ，评价指标为：

$$\text{recall@100}(q) = \frac{|R_{100}(q) \cap G_{100}(q)|}{100}, \quad (2)$$

其中 $R_{100}(q)$ 是算法返回集合， $G_{100}(q)$ 是 ground truth。正式最优点均按 **recall@100** ≥ 0.95 下 **latency** 最小选择；quick 结果只用于参数筛选和趋势判断，最终阈值判断使用 full-data 结果。

1.2 quick 规模的解释

ARM/Kunpeng 侧采用“quick sweep + full validation”的两阶段流程。quick sweep 将 base 限制为 50k，query 限制为 300，但 ground truth 仍来自完整 DEEP100K 100k base。因此即使使用 Flat 精确扫描，也只能命中 ground truth 中落在前 50k base 的部分；当前 ARM quick 中 Flat raw recall@100 为 0.502733，构成该 reduced-base 设置下的上限。报告中 ARM quick 不用于证明最终 recall 阈值，而用于分析跨平台趋势、MPI 扩展性、通信占比、OpenMP 混合并行和 SIMD 可移植性。

这种处理保留了三类结论：x86 full 给出本地正式 recall-latency 最优选择；ARM quick 给出同一代码在 AArch64/NEON 环境下的相对性能趋势；ARM full qsub 对 quick 筛出的 HNSW 候选做最终阈值验证。为了避免混淆，下文双平台趋势图明确标注 x86 full 100k/1000 与 ARM quick 50k/300，ARM full 结果则用单独表格汇报。

2 MPI 任务分配与全局 top-k 合并

2.1 base-sharding 数据划分

实现选择按 base data 划分，而不是按 query 划分。rank 0 读取数据集并广播矩阵元信息，base data 按连续区间切成 P 份：

$$|X_r| \in \left\{ \left\lfloor \frac{N}{P} \right\rfloor, \left\lceil \frac{N}{P} \right\rceil \right\}, \quad (3)$$

其中 P 为 MPI ranks。每个 rank 的全局 id 可由 `global_begin + local_id` 得到，因此无需额外 id 映射。query 和 ground truth 被广播到所有 rank，保证所有 rank 对同一批 query 搜索自己的本地分片。

主要数据分发代码如下：

```
1 MPI_Bcast(meta, 4, MPI_UNSIGNED_LONG_LONG, 0, MPI_COMM_WORLD);
2 MPI_Bcast(query.data(), query.size(), MPI_FLOAT, 0, MPI_COMM_WORLD);
3 MPI_Bcast(gt.data(), gt.size(), MPI_INT, 0, MPI_COMM_WORLD);
4 MPI_Scatterv(root_base, counts, displs, MPI_FLOAT,
5             local_base.data(), counts[rank], MPI_FLOAT, 0, MPI_COMM_WORLD);
```

主程序的执行流按“解析参数、初始化 MPI 线程级别、分发数据、本地建索引、批量查询、候选回收、全局合并、评价 recall”组织。MPI 初始化使用 `MPI_Init_thread`，根据命令行选择 FUNNELED 或 MULTIPLE；当前核心搜索线程不直接进入 MPI，因此默认结构符合 FUNNELED 的通信边界。数据加载只发生在 rank 0，其余 rank 通过广播获得 query 和 ground truth，通过 `MPI_Scatterv` 获得本地 base 分片。这样既避免每个进程重复读取完整数据，也使本地索引构建严格绑定到自己的数据分片。

实现上，参数解析在进入 `MPI_Init_thread` 前完成，保证每个 rank 使用相同的算法、通信、线程和 kernel 设置。数据分发阶段只把完整 base 保留在 root 临时对象中，`MPI_Scatterv` 完成后非 root 从未持有全量 base，root 也只在分发阶段需要完整输入。每个 rank 的本地分片使用连续 `std::vector<float>` 保存，后续 Flat 顺序扫描、IVF 建倒排链和 HNSW 插入都直接以 `base + local_id * dim` 定位向量。这个布局避免了跨 rank 共享指针，也让同一套本地索引接口能够在 Windows/MS-MPI 和 Linux/OpenMPI/MPICH 环境中复用。

如果采用 query-sharding，每个进程只处理一部分 query，但必须复制完整 base 或完整索引，否则无法得到全局 top-k。复制索引会增加内存占用，并且多个进程重复搜索相同数据结构，不适合体现 MPI 数据并行。base-sharding 的代价是每个 query 需要汇总多个 rank 的局部候选，但结果语义与单机全量检索一致。

数据划分采用连续均分，而不是随机划分或启发式划分。连续划分的优点是每个 rank 的全局 id 区间可以直接由 `global_begin` 推出，分发时只需要一次 `MPI_Scatterv`，也便于检查各 rank 数据量是否均衡。随机划分可能改善某些分布偏斜数据上的局部索引质量，但需要额外保存 id 映射并打乱连续内存访问；启发式划分如按 centroid 或图社区划分可能降低无效搜索，但需要全局训练或预处理，通信和实现复杂度更高。DEEP100K 在当前规模下连续均分已经使各 rank 数据量几乎一致，因此本文将随机/启发式划分作为复杂性对照，而不作为默认实现。

从复杂度看，base-sharding 将扫描型检索的主计算从 $O(Nd)$ 降为每 rank 约 $O(Nd/P)$ ，但引入 query 广播和候选回收。由于 query 矩阵规模为 $n_q d$ ，广播量与 base 无关；候选回收规模为 $O(Pn_q k)$ ，与返回候选数和 rank 数线性相关。对 DEEP100K 的 $d = 96, k = 100$ 而言，通信负载并不大，真正容易增长的是 root 对 $P \times k$ 候选的合并和同步等待。因此任务划分的核心矛盾不是“能否减少计算”，而是“减少的本地搜索时间是否大于新增的通信和 merge 时间”。

也可以设想按矩阵维度做 vertical split，即每个 rank 只保存向量的部分维度并计算局部内积，然后通过 `MPI_Reduce` 汇总完整距离。该策略能保持每个 rank 都看到全量候选，但每个 query 对所有 base 点都需要跨进程规约，通信量接近 $O(Nn_q)$ ，远大于 top-k 候选回收，不适合本实验规模。按 inverted list 或 centroid 划分则更接近全局 IVF：每个 rank 负责部分簇，query 只访问被选中的簇。它能减少无关 rank 的工作，但必须同步全局 centroid，并处理簇长度不均导致的负载倾斜。本文选择 base-sharding，是在实现稳定性、结果一致性和 MPI 通信代价之间的折中。

2.2 局部候选与全局 merge

每个 rank 对每个 query 维护一个固定容量 top-100 最大堆。新候选距离小于堆顶时替换堆顶；搜索结束后将堆转换为定长数组 `local_dist[nq*k]` 和 `local_ids[nq*k]`。定长布局使 `MPI_Gather`、`MPI_Igather`、p2p 和 one-sided 版本可以共享结果格式。

局部 top-k 的维护使用“距离越小越优”的最大堆：堆顶保存当前最差候选，插入新候选时只需与堆顶比较。堆大小固定为 $k = 100$ ，因此每个候选的维护代价为 $O(\log k)$ ，且 k 很小。搜索结束后，候选会被展开为连续数组而不是直接发送 C++ 容器，这是因为 MPI 只能高效传输连续内存。距离

数组和 id 数组分开发送，可以避免自定义 MPI 结构体带来的 padding 和跨平台对齐问题，也便于 one-sided 通信直接按偏移写入 root 窗口。

`heap_to_fixed_arrays` 会把每个 query 的局部堆排序后写入固定长度段；若某个局部索引返回不足 100 个候选，则用 `inf` 距离和无效 id 填充。root 合并时会跳过无效 id，因此所有通信分支都能使用同样的 `nq*k` 发送计数。这样牺牲了一点填充空间，但换来了稳定的 MPI 调用参数和统一的 recall 计算流程。对本文的 $n_q = 1000, k = 100, P \leq 8$ 来说，候选数组大小远小于 base/query 数据，固定布局的额外空间可以接受。

Algorithm 1 MPI base-sharding ANN 查询流程

- 1: rank 0 loads base/query/ground truth.
 - 2: Broadcast query and metadata; scatter base shards to ranks.
 - 3: Each rank builds local IVF or HNSW index.
 - 4: **for** each query q **do**
 - 5: Each rank searches only its local shard and keeps local top-100.
 - 6: **end for**
 - 7: Gather local candidates to rank 0.
 - 8: rank 0 merges $P \times 100$ candidates per query into global top-100.
 - 9: rank 0 evaluates recall@100.
-

全局 merge 对每个 query 处理 $P \times k$ 个候选，复杂度为 $O(Pk \log k)$ 。由于 $k = 100$ 固定，merge 的绝对成本不高，但当本地搜索被 MPI 和 SIMD 明显压缩后，merge 会成为越来越明显的串行部分。端到端延迟可以近似拆为

$$T(P) \approx \max_r T_{\text{local},r} + T_{\text{comm}}(Pn_q k) + T_{\text{merge}}(Pn_q k),$$

其中第一项随 P 增大下降，后两项随 P 增大上升。若本地索引是 Flat 或 IVF， $\max_r T_{\text{local},r}$ 占主导，MPI 分片收益明显；若本地索引已是 HNSW，单 rank 搜索已经很短，增加 rank 后更容易被候选合并和同步抵消。这个模型解释了为什么 HNSW 的全局最低点出现在 P1T4，而 MPI 多进程条件下的最优点是 P2T4。

一致性由两层保证。第一，rank 内返回的 id 是全局 id，因此不同 rank 的候选可以直接比较和合并；第二，所有 rank 返回 top-100 而不是变长候选，保证任何落在某个 rank 本地 top-100 内的真实近邻不会在通信前被截断。对于 Flat，这等价于精确全局 top-100；对于 IVF/HNSW，误差只来自本地 ANN 搜索没有访问到某些候选，而不是 MPI merge 过程改变了距离排序。

阶段计时把 build、distribute、search、comm、merge 和 total 分开。search、comm 等本地阶段先在各 rank 计时，再用 `MPI_Reduce` 取最大值作为该阶段的并行关键路径；merge 只在 root 发生，因此直接使用 root 端耗时。这样处理比简单平均各 rank 时间更接近用户实际等待时间，也能暴露某个 rank 较慢导致的尾部等待。负载均衡用各 rank 搜索工作量最大值与最小值之比衡量，用于判断瓶颈来自数据不均还是来自通信和合并。

每组实验可重复运行多次并取端到端 latency 的稳定最优值，以降低 Windows 后台调度和集群共享环境对单次测量的扰动。build 时间不参与每 query latency 的筛选；这是因为实验关注在线查询吞吐，但构建成本仍用于分析 HNSW、IVF 和 IVF-HNSW 在离线索引阶段的差异。

3 本地索引与混合并行实现

3.1 Flat 与 SIMD 距离核

Flat 是精确扫描基线。每个 rank 只扫描自己的本地分片，距离计算调用 `ann::ip_distance`。程序通过 `--kernel` 控制距离核：

- `scalar`: 禁止自动向量化的标量内积；
- `auto`: 编译器自动向量化；
- `simd`: x86 选择 AVX/AVX2, ARM 选择 NEON unroll4, 其余平台回退到自动向量化。

x86 Windows 编译使用 `/O2 /arch:AVX2 /openmp`; Linux/ARM 使用 `-O3 -fopenmp -march=native`。这使 MPI 主程序在不同平台上复用同一套搜索逻辑，只改变编译器和指令后端。

Flat 的实现故意保持简单：外层遍历 query，内层顺序扫描本地 base，候选进入固定大小堆。这个版本用于三个目的。第一，它验证 MPI 分片和全局 merge 是否正确，因为 Flat 在 full base 上应达到 `recall@100=1.0`；第二，它给 SIMD 距离核提供最纯粹的压力测试，绝大部分时间都花在内积；第三，它为 IVF/HNSW 的近似结果提供延迟和召回参照。优化过程中先保证 Flat 的分片与 merge 正确，再把同一套候选格式复用到 IVF 和 HNSW，降低后续算法分支出错的可能性。

距离核在接口层只暴露 `ann::ip_distance`，上层索引不需要关心 AVX2、NEON 或标量实现。x86 的手写 AVX 路径按 8 个 float 一组累加，ARM NEON 路径按 4 个 float 一组并做 unroll，尾部维度回退到标量循环。由于 DEEP100K 维度为 96，主循环基本不受尾部处理影响。这个封装使 Flat、IVF centroid 选择、IVF 链内扫描和 IVF-HNSW 的 probe 选择使用相同距离语义，避免不同算法分支因 kernel 不一致产生不可比较的 recall。

3.2 IVF 本地倒排索引

IVF 在每个 rank 的本地分片上独立构建 k-means centroid 和 inverted lists。搜索时先计算 query 到本地所有 centroid 的距离，选择最近的 `nprobe` 个倒排链，然后扫描链内向量并维护局部 top-100。单 rank 的主要计算量可近似写作：

$$O(nlist \cdot d + nprobe \cdot \bar{L} \cdot d), \quad (4)$$

其中 $\bar{L}$ 为本地倒排链平均长度。MPI 分片后 $\bar{L}$ 近似降为单机的 $1/P$ ，因此 IVF 对 ranks 增加较敏感。实现中没有跨 rank 同步全局 centroid，这减少了构建通信，但局部簇边界在各 rank 中不同；该设计更接近分布式局部索引，而不是全局 IVF 的并行扫描。

局部 IVF 的优势是构建与查询都只依赖本地数据，MPI 通信只发生在输入分发和候选回收阶段；缺点是各 rank 的 centroid 不共享，`nprobe` 的语义变为“每个局部索引访问 `nprobe` 个簇”。因此当 rank 数增加时，总访问簇数约为 $P \cdot nprobe$ ，但每个簇更短。实验中 IVF P8 `nprobe=32` 成为最优点，说明短倒排链带来的本地搜索下降超过了访问更多局部簇和 merge 的成本；继续增大 `nprobe` 后 recall 接近饱和，扫描量和堆维护反而成为主要开销。

IVF 构建采用本地 k-means：先在本地分片上初始化 centroid，再迭代分配样本和更新中心。由于每个 rank 的训练样本不同，centroid 只代表本地分布；查询时每个 rank 独立选取自己的最近簇，并把最终候选交给 root 合并。实现上，倒排链保存本地向量 id，搜索阶段再转换为全局 id。这个设计牺牲了全局 centroid 的一致性，但避免每轮 k-means 都进行跨 rank all-reduce，适合本实验强调的查询并行和通信方法对比。优化过程中的关键参数是 `nprobe`：它直接控制候选覆盖面，是 IVF recall-latency 曲线的主轴。

`top_probe_ids` 先计算 query 到本地所有 centroid 的距离，再用 `nth_element` 取前 `nprobe` 个簇，最后对 probe 结果排序。相比完整排序，`nth_element` 把 centroid 选择从 $O(nlist \log nlist)$ 降到平均 $O(nlist)$ ，在 `nlist=512` 时虽然不是主耗时，但能减少每个 query 的固定开销。链内扫描仍使用连续的 `local_base`，倒排链只保存 `uint32_t` 本地 id，因此候选访问不需要复制向量。`nprobe` 增大时，被访问倒排链数量和候选扫描量同步上升，这也是 IVF 延迟随 `nprobe` 近似线性增长的直接原因。

3.3 HNSW、multi-HNSW 与 IVF-HNSW

HNSW 路径在每个 rank 的本地分片上构建一个图索引。multi-HNSW 是该结构在 MPI 数据分片下的直接解释：多个 rank 各自维护一个 HNSW 子图，query 同时搜索多个子图后合并 top-k。IVF-HNSW 先构建本地 IVF，再在每个倒排簇内构建 HNSW；搜索时先用 centroid 选择簇，再在多个簇内图搜索。

图索引的查询开销远低于线性扫描，但构建开销更高，且 query 内部图遍历存在依赖，不适合轻易拆成细粒度并行。本文采用 query-level 并行，让同一 rank 内多个 query 并行进入 HNSW，避免在单 query 的图遍历步骤上引入锁和同步。

HNSW 的查询路径由当前入口点和候选优先队列逐步决定，后一步依赖前一步访问到的邻居集合。若将单 query 内的边或点强行并行化，需要多个线程共享 visited 集合和候选队列，锁竞争会发生在最频繁的数据结构上；若复制 visited 集合，则不同线程会重复扩展相同区域，候选一致性也需要额外合并。multi-HNSW 避开了单图内部同步，把并行粒度放到多个分片图之间，语义更简单，但代价是每个分片图都要独立搜索并返回候选。

图索引实现分为三条路径。直接 HNSW 在每个 rank 的分片上构图，查询返回本地 top-100；当 $P = 1$ 时它就是单图 HNSW，当 $P > 1$ 时多个局部图并行搜索后合并。multi-HNSW 强调“多个分片图共同提供候选覆盖”，用于观察数据集划分后图索引的 recall 提升和 merge 成本。IVF-HNSW 则先用 IVF 缩小候选簇，再在簇内使用 HNSW，希望把簇过滤和图搜索结合起来。实验结果表明该组合在当前规模下常数开销偏大，说明优化不能只叠加索引结构，还要看每层过滤是否真正减少了足够的距离计算或图访问。

HNSW 构建阶段把 label 直接设置为全局 id，因此 root 合并时不需要再做局部 id 到全局 id 的映射。索引构建完成后查询阶段不再插入或删除点，OpenMP/std::thread 只执行只读搜索；`ef` 在 batch 搜索前统一设置，避免每个 query 在线程内部频繁修改图索引参数。IVF-HNSW 中每个非空倒排簇都会创建一个小 HNSW，查询时对 `nprobe` 个簇分别调用 `searchKnn` 并再做本地 top-100 合并。这个实现能直接反映嵌套结构的真实成本：簇选择、多个小图入口、多个候选堆和最终 MPI merge 都会计入端到端 latency。

3.4 OpenMP 与 std::thread

进程内并行由 `parallel_for_queries` 封装。OpenMP 版本使用 dynamic schedule：

```
1 #pragma omp parallel for num_threads(threads) schedule(dynamic)
2 for (long long qi = 0; qi < (long long)nq; ++qi) {
3     search_one_query(qi);
4 }
```

C++ 标准线程版本按 query id 交错分配任务。IVF 的每个 query 访问候选数较稳定，线程调度开销相对显著；HNSW 的图遍历步数随 query 波动，dynamic schedule 更容易改善尾部等待。

线程模型上, OpenMP 用于批量 query 的循环调度, 适合通过编译选项快速切换线程数; `std::thread` 用作显式线程管理基线, Linux/AArch64 环境下由标准库落到 POSIX thread 运行时, Windows 环境下由 C++ 运行时映射到系统线程。两者都保持“工作线程只执行本地搜索、主线程执行 MPI 通信”的结构, 因此不会改变 MPI 通信语义, 也便于单独比较线程调度代价。

线程管理代价主要由三部分组成: 线程创建和销毁、任务调度、以及多个线程同时访问本地索引造成的 cache 竞争。OpenMP 在循环区域内复用线程池, 适合本文这种 batch query; `std::thread` 版本显式创建固定数量线程并交错分配 query, 调度逻辑可控, 但缺少 OpenMP dynamic schedule 对尾部任务的自动平衡。HNSW 单个 query 的访问步数波动大, dynamic schedule 的收益更明显; IVF 每个 query 的 nprobe 固定, 任务时间更均匀, 过多线程时调度收益会被 cache 和内存带宽竞争抵消。

线程边界被限制在本地 batch search 内部: 每个线程只写自己负责 query 对应的连续候选区间, 即从 $qi \cdot k$ 起始的 100 个距离和 id, 不共享候选堆, 也不直接调用 MPI。这样可以避免候选堆加锁, 并让 FUNNELED 线程级别满足主流程需求。对 HNSW 来说, 线程共享的是构建后的只读图结构; 对 IVF 来说, 线程共享 centroid、倒排链和 base 数组, 但每个 query 的 probe 列表、visited 计数和 top-k 堆都是线程局部对象。

3.5 MPI 通信模式

局部结果回收实现了四种方式:

- **blocking**: 使用 `MPI_Gather` 收集距离和 id;
- **nonblocking**: 使用两次 `MPI_Igather` 后 `MPI_Waitall`;
- **point-to-point**: 非 root rank 使用 `MPI_Send`, root 使用 `MPI_Recv`;
- **one-sided**: root 创建 `MPI_Win`, 其他 rank 使用 `MPI_Put` 写入窗口。

程序默认请求 `MPI_THREAD_FUNNELED`; 额外实验请求 `MPI_THREAD_MULTIPLE`。当前设计中工作线程不直接调用 MPI, MULTIPLE 主要用于测量运行时线程安全支持的额外影响, 而不是用于流水线通信。

四种通信方式共享相同的候选布局, 因此差异只来自 MPI API 的进度模型和同步方式。blocking 版本简单稳定, 所有 rank 在 gather 调用处同步; nonblocking 版本允许提前发起传输, 但如果没有后续可重叠计算, 最终仍会在 wait 处同步; p2p 版本暴露了 root 逐个接收的顺序, 便于实现自定义树形合并; one-sided 版本减少了接收端显式调用, 但窗口创建、fence 和内存同步本身也有固定成本。实验保留这些版本, 是为了比较 MPI 编程方法对 ANN top-k 回收阶段的实际影响。

通信分支只替换“局部候选回收到 root”的阶段, 不改变本地搜索和 root merge。这样做能保证不同通信 API 的实验可比: 输入候选相同、候选数量相同、merge 逻辑相同。blocking 版本使用两次 gather 分别收集距离和 id; nonblocking 版本发起两次 igather 并统一等待; p2p 版本让非 root 主动发送, root 按 rank 接收; one-sided 版本由 root 创建窗口, 其他 rank 通过 put 写入对应偏移。所有分支结束后 root 得到相同形状的候选矩阵, 因此后续 recall 和 latency 差异可以归因于通信阶段, 而不是算法分支改变了候选质量。

从实现角度看, 通信阶段有两个必须保持不变的约束。第一, 距离和 id 必须按同一 rank、同一 query、同一候选位置对齐, 否则 root merge 会把距离与错误 id 配对; 因此所有分支都采用 $(rank \cdot nq + qi) \cdot k + j$ 的偏移公式。第二, root 必须在 merge 前确认所有 rank 的候选写入完成; blocking 和 p2p 由调用返回保证, nonblocking 由 `MPI_Waitall` 保证, one-sided 由第二轮 `MPI_Win_fence` 保证。这些同步点也是通信优化难以绕开的根本原因。

4 实验设置

4.1 平台与数据

表 1: 实验平台与规模

平台	编译/运行环境	数据规模	说明
x86 full	Windows + MSVC 19.43 + MS-MPI 10.1	100k base / 1000 query	正式 recall@100 阈值筛选
ARM quick	ARM/Kunpeng + mpic++ + OpenMP	50k base / 300 query	reduced-base 趋势与可移植性分析
ARM full qsub	ARM/Kunpeng + PBS/qsub + mpic++ + OpenMP	100k base / 1000 query	ARM 最终候选全量验证

x86 正式命令为:

```
1 cd ann
2 powershell -NoProfile -ExecutionPolicy Bypass -File .\run_mpi_x86.ps1 -Clean
```

ARM/Kunpeng quick 命令为:

```
1 cd /home/$USER/ann
2 ANN_DATA=/anndata bash run_mpi_arm_kunpeng.sh --quick
```

ARM/Kunpeng 全量验证命令为:

```
1 cd /home/$USER/ann
2 ANN_DATA=/anndata bash run_mpi_arm_complete.sh
```

集群提交使用 `qsub_mpi.sh`, 不使用 `test.sh`。脚本中项目目录为 `ann`, 编译器为 `mpic++`, 默认资源为 `nodes=1:ppn=8`, 运行时 `NP=2`、每进程 `THREADS=4`, 满足 `np <= nodes*ppn`、`nodes <= 4`、`ppn <= 8`。qsub 用于 HNSW P2T4/P4T2/P8T1 等全量验证配置, 提交命令为:

```
1 cd ann
2 qsub qsub_mpi.sh
```

脚本会在主节点编译可执行文件, 将可执行文件、HNSW 头文件目录和数据复制到计算节点, 通过 `mpiexec -np` 启动程序, 结束后把输出目录复制回主节点。本程序运行不依赖预置的 `files` 输入目录; `files` 主要作为输出目录回传。若作业长时间无输出, 可用 `qstat` 检查队列状态; 若确认死锁或资源长期占用, 则使用 `qdel` 删除对应作业。

4.2 实验矩阵

x86 full 与 ARM quick 的实验结果规模分别为:

- x86 full: 100k base / 1000 query, 共 172 组配置;
- ARM quick: 50k base / 300 query, 共 111 组配置;
- ARM full qsub: 100k base / 1000 query, 对 quick 筛出的 HNSW 候选进行 1/2 节点、2/4/8 ranks、1/2/4 threads 的全量验证, 并补充 perf profiling。

实验矩阵覆盖内容如下:

- 算法: Flat、IVF、HNSW、multi-HNSW、IVF-HNSW;
- MPI ranks: x86 覆盖 1/2/4/8, ARM quick 主扫描覆盖 1/2/4, 图索引补充 8 ranks;
- OpenMP threads: x86 覆盖 1/2/4, ARM quick 覆盖 1/2;
- IVF nprobe: x86 覆盖 16/32/64/128/256, ARM quick 覆盖 16/32/64;

- 通信方法: blocking、nonblocking、p2p、one-sided;
- 距离核: scalar 与 SIMD;
- profiling 阶段: build、distribute、search、comm、merge、total、work imbalance。

实验顺序先覆盖不同 base 规模下的 Flat/IVF 和 nprobe, 再固定 100k base 比较通信方式、OpenMP threads、`std::thread`、`MPI_THREAD_MULTIPLE`、scalar/SIMD 和图索引组合。x86 full 默认每组重复 3 次并取稳定最低 latency; ARM quick 为了降低集群占用采用单次运行, 保留相同算法和并行维度, 便于观察跨平台趋势。ARM full qsub 不再重复大范围 sweep, 而是把 quick 与 x86 full 共同指向的 HNSW P2T4/P4T2/P8T1 候选放回 100k/1000 数据规模验证, 最终按 $\text{recall}@100 \geq 0.95$ 下 latency 最小选择。

5 总体结果与跨平台趋势

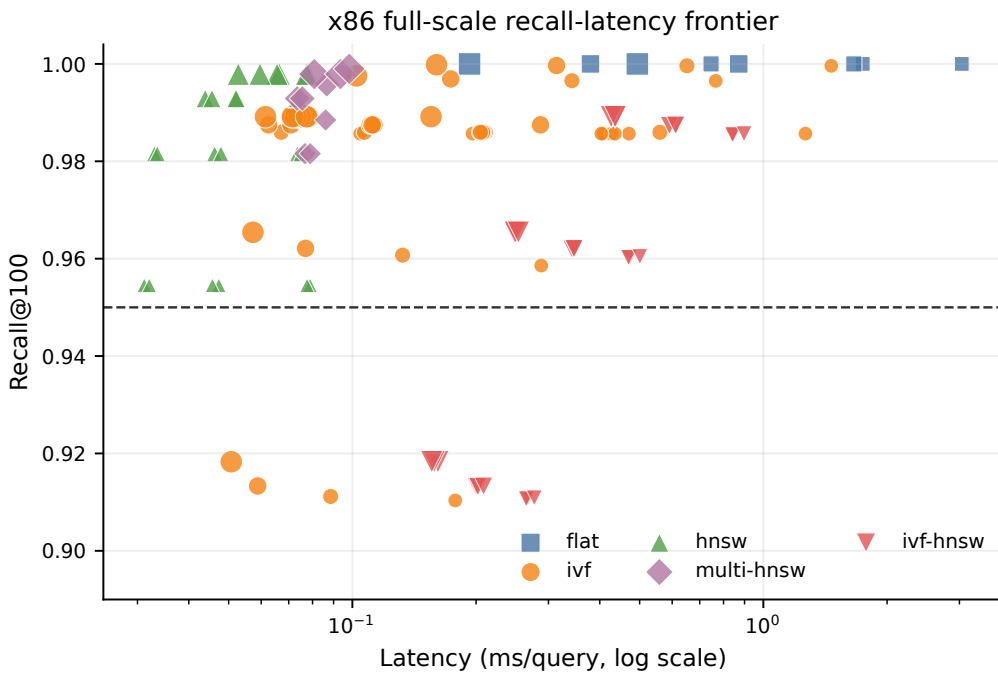


图 1: x86 full 100k/1000-query 实验的 recall-latency 分布。虚线为 $\text{recall}@100=0.95$ 。

图 1 显示 x86 full 下的正式筛选空间。Flat 全部位于 $\text{recall}=1.0$, 但延迟高; IVF 形成由 nprobe 控制的连续曲线; HNSW 和 multi-HNSW 位于左上区域; IVF-HNSW 可达到阈值, 但延迟明显高于直接 HNSW。该图中的最优选择不是 recall 最大的点, 而是 $\text{recall}@100$ 达到 0.95 后的最低 latency 点。

从算法族看, Flat 的意义是给出精确基线和 raw recall 上界; 它在 P8 下仍为 0.1928 ms/query, 说明单纯扫描即使用 SIMD 和 MPI 分片也难以达到图索引的延迟。IVF 是最清晰的 recall-latency 连续控制器, nprobe 从 16 增加到 32 时跨过阈值, 之后继续增大主要换来小幅 recall 增益。HNSW 则直接把访问点数降到很低, 因此位于 Pareto 前沿左端。multi-HNSW 的 recall 更高但延迟高于 HNSW, 说明“分片覆盖更多候选”与“多图重复搜索”之间存在明显 trade-off。

优化过程先从 Flat-SIMD 验证正确性和基础扩展性开始。x86 full 中 Flat-SIMD 从 P1 的 1.7449 ms/query 降到 P8 的 0.1928 ms/query, 说明 base-sharding 和全局 merge 的语义正确且能显著减少扫描时间。随后引入 IVF 后, P8 nprobe=32 把 latency 降至 0.0573 ms/query, 并保持

recall=0.9654；再引入 HNSW 后，P1T4 达到 0.0312 ms/query。这个顺序表明主要收益经历了三次瓶颈转移：从全量距离计算到倒排候选覆盖，从倒排扫描到图访问数量，再从本地搜索转向线程调度、通信和 merge。

表 2: x86 full 中 recall@100 $\geq$ 0.95 下各算法族最优配置

算法	ranks	threads	nprobe	ef	线程模型	通信	latency	recall
HNSW	1	4	64	64	OpenMP	nonblocking	0.0312	0.9545
IVF	8	1	32	64	stdthread	blocking	0.0573	0.9654
multi-HNSW	4	1	64	32	stdthread	nonblocking	0.0734	0.9929
Flat	8	1	512	64	stdthread	blocking	0.1928	1.0000
IVF-HNSW	8	1	32	64	stdthread	nonblocking	0.2488	0.9654

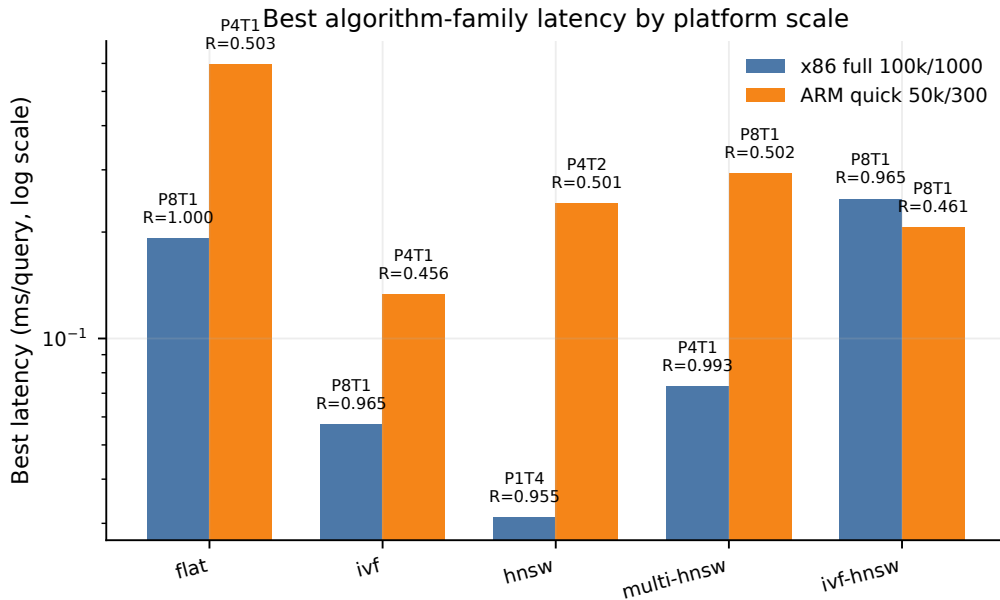


图 2: x86 full 与 ARM quick 的各算法族最低延迟。ARM quick 的 recall 是 50k base 对 100k ground truth 的 raw recall。

图 2 不把两个平台视作同规模排名，而用于观察同一实现的趋势。ARM quick 的 Flat raw recall 为 0.5027，HNSW/multi-HNSW 接近该上限，说明图索引在 reduced-base 内部能覆盖大部分可命中真近邻；IVF 在 nprobe=16 时延迟最低但 raw recall 只有 0.4562，nprobe=64 时 raw recall 升至 0.4953。x86 full 中 HNSW 是全局最低延迟点；ARM quick 中 IVF 的最低延迟较低，是因为 nprobe=16 的搜索宽度很窄，不能与 x86 full 的 recall 阈值结果直接等价。

表 3: ARM quick 的 reduced-base raw recall 解释

算法	raw recall 上限/最高值	相对 Flat 上限	代表 latency
Flat	0.502733	1.000	0.5987
IVF	0.495333	0.985	0.2929
HNSW	0.501500	0.998	0.3336
multi-HNSW	0.502633	1.000	0.3858
IVF-HNSW	0.497767	0.990	0.6461

表 3 说明 ARM quick 的 raw recall 上限由 reduced-base 设置决定。HNSW 与 multi-HNSW 的最高 raw recall 已接近 Flat 上限，算法质量在 quick 内部是合理的；IVF-HNSW 的 raw recall 也接近上限，但延迟较高，原因仍是多簇图搜索带来较大常数开销。

ARM quick 的更合理读法是归一化到 Flat 上限。IVF 最高 raw recall 为 0.4953，达到 Flat 上限的 98.5%；HNSW 和 multi-HNSW 分别达到约 99.8% 和 100.0%。这说明 ARM 侧没有出现算法失效，低于 0.95 是数据规模与 ground truth 不一致造成的评价上限问题。跨平台对比因此不能把 ARM quick 的 raw recall 与 x86 full 的正式 recall 混排，只能比较趋势：nprobe 增大时 recall 上升、ranks 增大时搜索缩短、通信占比随本地搜索缩短而上升。

表 4: ARM/Kunpeng full-data qsub 验证：HNSW + OpenMP + SIMD + nonblocking

节点	ranks	threads	latency	recall	build(s)	search	comm	merge
1×8	2	4	0.1318	0.9814	20.75	106.7	2.1	23.9
1×8	4	2	0.2512	0.9930	9.80	220.3	36.1	29.9
2×4	4	2	0.2188	0.9930	8.92	189.5	25.7	28.9
2×8	8	1	0.4315	0.9979	4.22	393.3	48.9	35.8

表 4 是 ARM 平台上用于最终判断的 full-data 结果。四个 qsub 配置均达到 $\text{recall}@100 \geq 0.95$ ，其中 1 节点、2 ranks、每 rank 4 个 OpenMP 线程的 P2T4 配置延迟最低，为 0.1318 ms/query、 $\text{recall}@100=0.9814$ 。P4T2 和 P8T1 的 recall 更高，但延迟显著上升，说明在 HNSW 已经降低访问点数后，继续增加 rank 会让多图搜索、候选回收和 root merge 的成本超过分片收益。2 节点 P4T2 比 1 节点 P4T2 更快，表明跨节点资源增加可以缓解部分本地搜索压力；但它仍慢于 1 节点 P2T4，说明当前规模下跨节点通信和更多分片图的额外开销没有被更高并行度抵消。

将同一 HNSW P2T4 路径放在 x86 与 ARM full-data 上比较，x86 full 的 0.0330 ms/query 与 ARM qsub 的 0.1318 ms/query 相差约 4.0×。两者 recall 几乎一致，差异主要来自单核频率、SIMD 宽度、OpenMP 运行时、MPI 实现以及 HNSW 随机访存对 cache/内存层次的敏感性。这个结果也解释了为什么 ARM quick 可以用于筛选趋势，却必须用 full-data qsub 做最终确认：quick 的 reduced-base raw recall 不能判断阈值，但它筛出的 rank-thread 方向在 full-data 上仍成立。

6 IVF 参数消融与扩展性

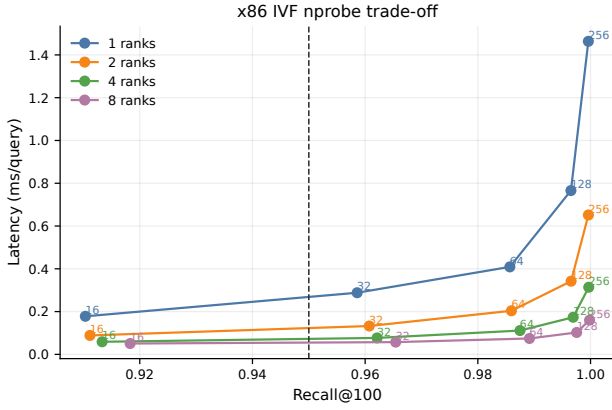


图 3: x86 full 中 IVF nprobe 的 recall-latency trade-off, 点旁数字为 nprobe。

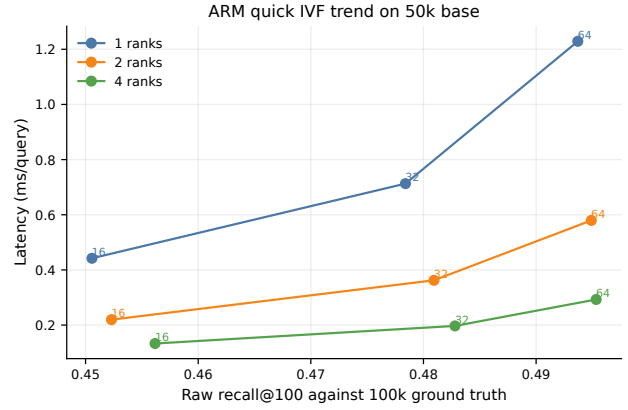


图 4: ARM quick 中 IVF nprobe 趋势。raw recall 对照 100k ground truth, 最高值受 50k base 限制。

x86 full 中, nprobe=16 在所有 ranks 下都低于 recall 阈值; nprobe=32 在 8 ranks 下达到 0.9654 recall 和 0.0573 ms/query, 是 IVF 的正式最优点; nprobe=64/128/256 继续提高 recall, 但 latency 增长更快。该趋势符合 IVF 的复杂度: 扫描链数增加会近似线性增加候选访问量, 而 recall 提升逐渐饱和。

更细地看, P1 从 nprobe=32 到 64 时 recall 从 0.959 提高到 0.986, 但 latency 从 0.288 增到 0.409 ms/query; P8 同样从 0.965 提高到 0.989, 但 latency 只从 0.057 增到 0.074 ms/query。rank 数越大, 单 rank 的链越短, 增加 nprobe 的绝对代价越低; 但 P8 已经把搜索阶段压得很短, 继续提高 nprobe 的收益主要是 recall 精修, 而不是降低延迟。因此正式最优点出现在刚跨过阈值的 P8 nprobe=32。

表 5: x86 full IVF nprobe 消融: latency / recall

ranks	nprobe=16	nprobe=32	nprobe=64	nprobe=128	nprobe=256
1	0.178/0.910	0.288/0.959	0.409/0.986	0.766/0.997	1.464/1.000
2	0.089/0.911	0.133/0.961	0.204/0.986	0.342/0.997	0.652/1.000
4	0.059/0.913	0.077/0.962	0.111/0.987	0.174/0.997	0.314/1.000
8	0.051/0.918	0.057/0.965	0.074/0.989	0.102/0.998	0.160/1.000

ARM quick 中, nprobe 从 16 增加到 64 时 raw recall 从约 0.451–0.456 提高到约 0.494–0.495, 已经接近 Flat 上限 0.5027; latency 则从 P4 的 0.1332 ms/query 增至 0.2929 ms/query。该趋势与 x86 full 一致: 更大的 nprobe 提高覆盖面, 但带来更多倒排链扫描。不同的是, ARM quick 中由于 base 只有 50k, raw recall 的绝对值被规模限制, 不能用 0.95 阈值筛选。

ARM 的 P1/P2/P4 曲线斜率还说明了另一个问题: 当 quick 模式把本地数据减半后, P4 下每 rank 的倒排链很短, nprobe=16 已能获得较低延迟; 但如果为了逼近 reduced-base 上限而使用 nprobe=64, latency 会明显上升。换言之, ARM quick 中 IVF 的最低延迟点和最高 raw recall 点并不相同, 这与 x86 full 中“刚达阈值最优”的选择逻辑一致, 只是 quick raw recall 上限不同。

7 通信方法、同步与 MPI 线程支持

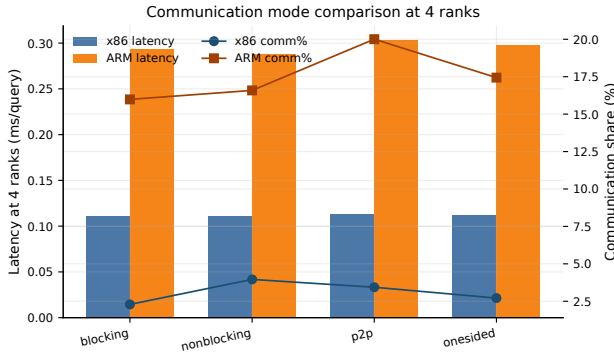


图 5: 4 ranks、IVF nprobe=64 下通信方式对 latency 和通信占比的影响。

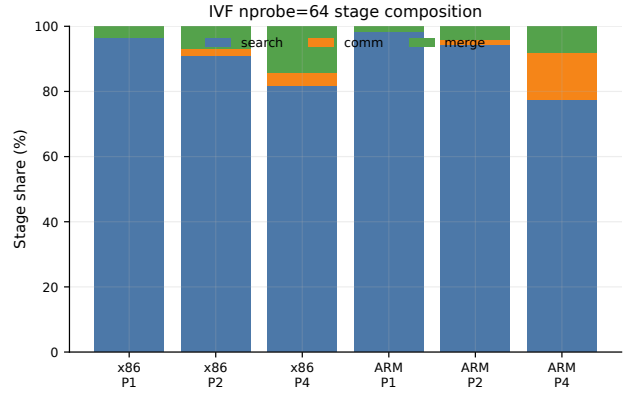


图 6: IVF nprobe=64 的 search、comm、merge 阶段占比。每个平台使用各 ranks 下最低 latency 的通信方式。

图 5 比较了四种结果回收方式。x86 full 中四种方式的 latency 均约为 0.110–0.113 ms/query，通信占比约 2.5%–4.1%。ARM quick 中 latency 约为 0.288–0.304 ms/query，通信占比约 15.7%–20.1%，明显高于 x86。这表明 reduced-base 场景下本地搜索更短，通信和同步开销更容易暴露；同时 ARM/Kunpeng 平台的 MPI 实现、进程间通信路径和内存系统也可能放大该占比。

表 6: IVF nprobe=64 通信方式对比：latency，括号内为总通信时间 ms

平台	ranks	blocking	nonblocking	p2p	one-sided
x86 full	1	0.409 (0.2)	0.404 (0.2)	0.409 (0.2)	0.428 (5.0)
x86 full	2	0.204 (4.2)	0.204 (0.4)	0.211 (2.0)	0.209 (3.4)
x86 full	4	0.111 (2.5)	0.110 (4.4)	0.113 (3.9)	0.112 (3.0)
x86 full	8	0.074 (10.2)	0.077 (8.5)	0.075 (6.6)	0.072 (5.9)
ARM quick	1	1.229 (0.1)	1.180 (0.2)	1.264 (0.2)	1.260 (0.6)
ARM quick	2	0.580 (5.9)	0.574 (20.0)	0.584 (0.6)	0.530 (2.4)
ARM quick	4	0.293 (14.0)	0.288 (14.3)	0.304 (18.2)	0.298 (15.6)

nonblocking 的理论优势在于通信与计算重叠。但当前实现是“先完成本地搜索，再统一 gather”，MPI_Igather 没有与下一批 query 的搜索重叠，因此只改变等待方式，不改变整体依赖链。真正利用 nonblocking 需要 query batch 流水线：搜索 batch b 的同时传输 batch $b - 1$ 的候选，并让 root 分批 merge。该方案会增加 buffer 管理和结果顺序控制，适合作为后续优化。

从结果看，x86 P4 下四种通信方式的 latency 差异只有约 0.003 ms/query，远小于 nprobe 或算法族差异；ARM P4 下通信占比更高，但不同通信 API 的总延迟仍集中在 0.288–0.304 ms/query。原因是候选回收的数据量固定，且 root merge 是所有方法共同的后处理。one-sided 不一定更快，因为 MPI_Win_create 和 fence 的同步成本在小消息场景下会很明显；p2p 也不一定更快，因为 root 顺序接收会暴露接收端热点。通信方法的主要价值不是直接替代算法优化，而是为后续树形 reduce 或流水线结构提供实现基础。

7.1 MPI_THREAD_MULTIPLE

x86 full 中, MPI_THREAD_FUNNELED 与 MPI_THREAD_MULTIPLE 的差异很小: 1/2/4/8 ranks 下 FUNNELED latency 分别为 0.4068、0.2040、0.1105、0.0779 ms/query; MULTIPLE 分别为 0.4035、0.2049、0.1118、0.0774 ms/query, MS-MPI 返回 provided=3。原因是当前线程只参与本地 query 搜索, MPI 调用仍由主线程执行。MULTIPLE 的价值在于多线程同时发起通信的流水线结构; 在本实现中, FUNNELED 更简单, 且不会引入额外线程安全管理成本。

这组结果还说明 MPI 线程支持不能脱离程序结构讨论。若使用 FUNNELED, MPI 库只需保证主线程进入 MPI; 若请求 MULTIPLE, MPI 库需要支持任意线程进入 MPI, 内部可能维护锁和进度状态。当前实验中 MULTIPLE 没有显著负担, 是因为真正进入 MPI 的仍然是主线程; 若将每个 OpenMP 线程直接发送自己的局部候选, MULTIPLE 的成本和收益才会同时出现。报告中因此把 MULTIPLE 视为编程模型对比, 而不是默认加速策略。

8 阶段 profiling、负载均衡与 cache locality

图 6 展示了阶段比例。x86 full 中, P1/P2/P4 的 search 占比分别约 96.5%、90.9%、81.8%, merge 占比分别约 3.4%、7.1%、14.4%。ARM quick 中, P1/P2/P4 的 search 占比分别约 98.2%、94.3%、77.5%, P4 下通信占比上升到约 14.3%。这说明 MPI 扩展的瓶颈会随进程数从本地搜索转向通信和 merge; ARM quick 因问题规模更小, 本地搜索被切短后通信占比更快上升。

阶段比例可以用 Amdahl 定律解释。若 search 阶段占比为 s , 理想地把 search 加速 P 倍后, 总加速上界约为 $1/((1-s) + s/P)$ 。P1 时 search 占比接近 96%, 继续并行化有较大空间; P4 后 x86 merge 占比已超过 14%, 即使本地 search 继续变快, 串行 merge 和通信也会限制端到端收益。ARM quick 的 P4 search 占比更低, 说明同样的 MPI rank 数在小规模问题上更早进入通信/同步主导区间。

负载均衡方面, 连续 base 切分使每个 rank 的向量数最多相差 1。Flat 和 HNSW 的 `work_imbalance` 基本为 1.0; IVF 在 x86 最优点附近约为 1.03, 来自局部倒排链长度差异。这个数值说明当前主要瓶颈不是数据量不均, 而是 root merge 和候选传输的阶段比例。

cache locality 方面, MPI 分片能改善本地扫描数组和 top-k 堆的局部性。x86 full 中 Flat-SIMD 从 P1 的 1.7449 ms/query 降到 P8 的 0.1928 ms/query, 超过理想 8 倍加速, 说明分片后的工作集更容易留在 cache 中。IVF 同样受益于倒排链变短; 但 `nprobe` 过大时会访问许多短链, 链切换和堆维护的常数成本上升, 使 latency 与 `nprobe` 接近线性增长。

cache 优化不是单独的代码技巧, 而是由数据划分改变工作集大小。Flat 扫描按连续内存访问, 分片后每 rank 顺序读取更小的 base 区间, 硬件预取更稳定; IVF 访问倒排链, 链内局部性较好, 但多个链之间跳转会破坏连续访问。HNSW 的邻接访问更随机, cache 友好性弱于 Flat/IVF, 但访问点数少, 随机访存的总量被压低。由此可见, 不同索引的 cache 问题不同: Flat 依赖带宽和 SIMD, IVF 依赖链布局 and `nprobe`, HNSW 依赖减少图访问点数。

ARM 主节点上额外使用 `perf stat` 对 HNSW P2T4 的 full-data 配置做硬件计数器补充。该次 profiling 的 `recall@100=0.98135`, 程序侧 latency 为 0.1503 ms/query; `perf` 记录到约 1.109×10^{11} cycles、 1.287×10^{11} instructions, IPC 约 1.16, cache references 约 4.024×10^{10} , cache misses 约 5.947×10^8 , cache miss rate 为 1.478%。该计数器结果只用于解释指令与 cache 行为; 正式 ARM latency/recall 仍以 PBS qsub full-data 结果为准, 因为 `perf` 包裹和主节点运行环境会引入额外测量开销。与 qsub P2T4 的 0.1318 ms/query 相比, `perf` 包裹运行约慢 14%, 但 recall 完全一致, 说明差异来自测量环境而不是算法分支。IPC 只有 1.16, 而 cache miss rate 不高, 表明 HNSW full-data 查询

并非简单受最后一级 cache miss 率单独支配；分支判断、候选优先队列、邻接表间接访问和 OpenMP 调度共同限制了指令流水线利用率。

build 阶段也影响算法选择。IVF 的构建主要是本地 k-means 和倒排链填充，参数 nprobe 只影响查询，不需要重建索引；因此 IVF 适合做细粒度参数消融。HNSW 的 ef、M 和数据分片会影响图结构，构建时间明显高于 IVF，但查询阶段访问点数少。IVF-HNSW 同时支付 IVF 构建和多个簇内 HNSW 构建，若查询量不足以摊销 build 成本，端到端收益会进一步下降。报告的 latency 指标只统计在线查询，但工程选择仍要考虑索引更新频率。

9 OpenMP 混合并行与 SIMD

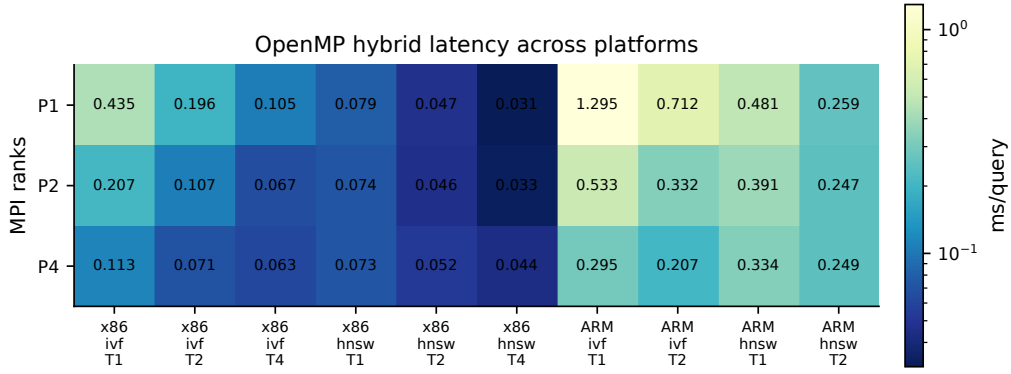


图 7: OpenMP 混合并行热力图。数值为 latency，行为 MPI ranks，列为平台、算法和线程配置。

图 7 显示，x86 full 中 IVF 从 T1 到 T4 持续收益，P1 下由 0.435 降至 0.105 ms/query；P4 下由 0.113 降至 0.063 ms/query。P8 下 T2 到 T4 的收益很小，说明 MPI 分片已使单 rank 工作量较小，线程调度和 cache 竞争开始抵消收益。HNSW 在 x86 中 P1T4 最快，P4T4 比 P2T4 慢，说明图搜索本身很短后，增加 ranks 会提高 merge 成本。

OpenMP 和 MPI 的组合存在资源分配边界。固定核心数下，增加 ranks 会减少每 rank 数据量，但也增加候选回收和进程调度；增加 threads 会提高单 rank 内 query 并发，但多个线程共享同一个本地索引和内存带宽。IVF 的 query 之间几乎独立，适合 OpenMP 批并行；HNSW 单 query 访问点少，T4 后继续增加 MPI rank 的收益很小。最终 P2T4 成为 MPI 多进程最优点，反映出“少量分片 + 足够进程内并发”比“很多分片 + 每分片很小任务”更适合该数据规模。

ARM quick 中 OpenMP T2 仍有明显收益：IVF P4 从 0.295 降至 0.207 ms/query，HNSW P4 从 0.334 降至 0.249 ms/query。由于 ARM quick 只覆盖到 T2，不能判断 T4 是否继续有效；但趋势表明 query-level 并行在 ARM 上同样可移植，收益主要取决于每 rank 剩余工作量是否足够大。

ARM full qsub 对这一判断做了更直接的验证。P2T4 在 100k/1000 数据上达到 0.1318 ms/query，而 P4T2 和 P8T1 分别退化到 0.2512 ms/query 和 0.4315 ms/query；也就是说，在同样 8 个硬件线程预算附近，把并行度更多分给 rank 内 OpenMP，比把数据拆成更多 HNSW 子图更有利。原因是 HNSW 的单 query 搜索路径较短，rank 数增大后每个局部图仍要执行入口搜索、候选队列维护和 top-100 返回，root 还要合并更多候选；OpenMP 则主要并行化 query batch，不改变全局候选矩阵的 rank 维度，因此 P2T4 成为 ARM 和 x86 上共同稳定的 MPI 多进程候选。

从 x86 的 rank-thread 组合看，HNSW P1T1/P1T2/P1T4 分别为 0.0789、0.0473、0.0312 ms/query，线程并行收益明显；P2T4 为 0.0330 ms/query，已经接近 P1T4；P4T4 和 P8T4 分别退化到 0.0438、

0.0596 ms/query，说明继续增加 ranks 后 merge 和进程调度超过了数据分片收益。IVF 的表现不同：P1T4 为 0.1045 ms/query，P4T4 为 0.0625 ms/query，P8T4 为 0.0615 ms/query，说明 IVF 仍能从 MPI 分片中受益，但 P8 后 OpenMP 的边际收益已经很小。这个差异来自算法内部访问量：IVF 仍有较多链内距离计算，HNSW 则更早进入固定开销主导区。

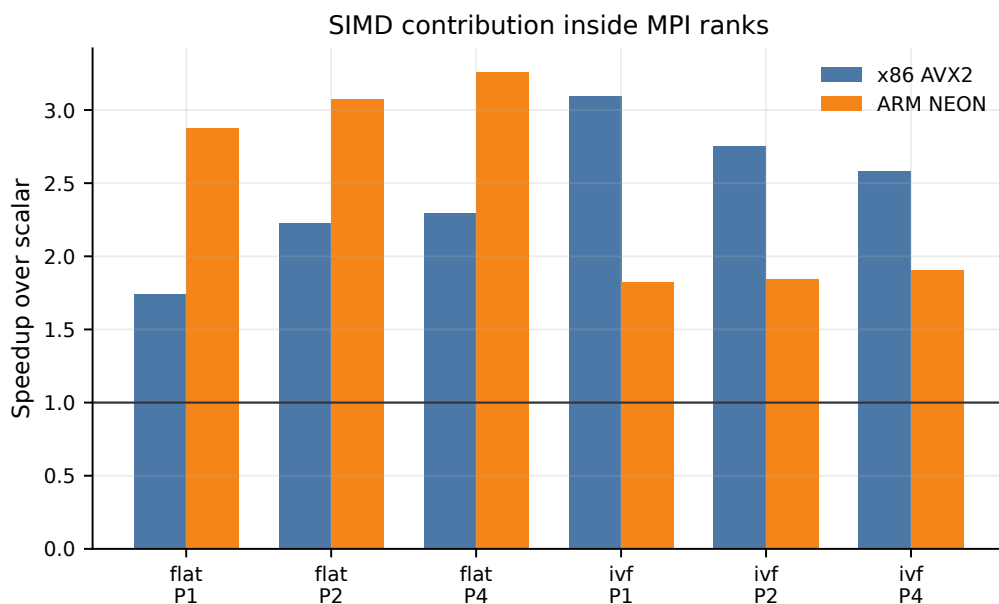


图 8: MPI rank 内 scalar 与 SIMD 距离核对比。x86 为 AVX2，ARM 为 NEON。

图 8 显示，ARM NEON 对 Flat 的加速比达到 2.88–3.26 $\times$ ，高于 x86 Flat 的 1.74–2.29 $\times$ ；IVF 中 x86 AVX2 加速比为 2.58–3.09 $\times$ ，ARM NEON 为 1.82–1.91 $\times$ 。差异来自两方面：Flat 的计算几乎全在内积核，ARM 标量基线较慢，因此 NEON 可见收益高；IVF 除内积外还有 centroid 选择、倒排链遍历和 top-k 堆维护，ARM 上这些非 SIMD 部分占比更高，导致 SIMD 加速比低于 Flat。该结果符合 Amdahl 定律：当通信、merge 和堆维护占比上升时，单纯加速距离核的收益会下降。

SIMD 对 MPI 的影响也具有边际递减。P1 时距离计算占比最高，向量化能直接降低主耗时；P4/P8 时每 rank 扫描量下降，通信、merge、堆维护和索引遍历占比上升，SIMD 加速比会被稀释。因此“MPI + SIMD”不是两种加速比简单相乘，而是不断改变瓶颈所在。报告保留 scalar/SIMD 对照的目的，就是区分距离核优化带来的收益和 MPI 分片带来的收益。

10 图索引组合与负优化分析

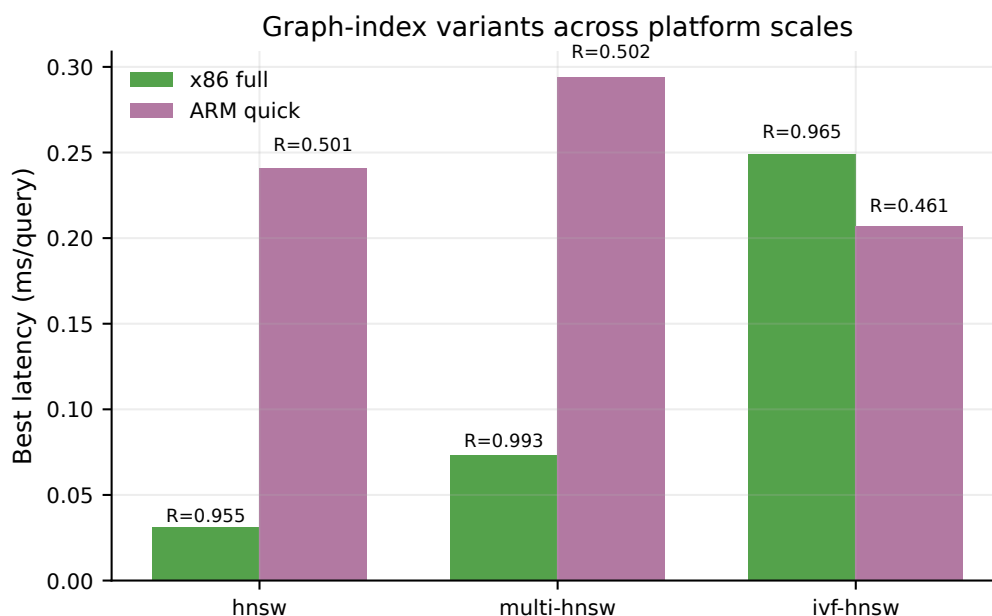


图 9: HNSW、multi-HNSW、IVF-HNSW 在 x86 full 与 ARM quick 中的对比。

x86 full 中, HNSW 最优为 0.0312 ms/query、recall=0.9545; multi-HNSW 为 0.0734 ms/query、recall=0.9929; IVF-HNSW 为 0.2488 ms/query、recall=0.9654。multi-HNSW 的 recall 更高, 是因为多个分片各自返回 top-100, 候选覆盖更大; 但每个 rank 都要执行本地图搜索, root 还要合并更多候选, 因此 latency 不如单 HNSW。IVF-HNSW 的延迟最高, 原因是查询先计算 centroid, 再对多个簇内 HNSW 重复发起图搜索, 小图搜索的固定开销和多簇调度抵消了 IVF 的过滤收益。

HNSW-on-HNSW 可以理解为先把数据分为多个底层 HNSW 子图, 再在子图入口或子图代表点之上建立上层导航结构。本文没有单独构建显式的上层 HNSW, 而是用 multi-HNSW 研究“多个底层 HNSW 并行搜索后合并”的主要代价: 每个子图都会产生一次图搜索和一组 top-100 候选, root 合并压力随子图数增加。若再加入上层 HNSW, 它可能减少需要访问的子图数量, 但会引入上层图构建、入口选择误差和跨子图候选一致性问题。因此本文把 multi-HNSW 与 IVF-HNSW 作为 HNSW 组合策略的可执行近似, 用实验结果分析这类结构的收益和负优化来源。

进一步看 ef 的作用, multi-HNSW 在 P4 ef=32 时已达到 0.0734 ms/query、recall=0.9929; 继续把 ef 提到 128 后 recall 只小幅上升, 而 latency 增至约 0.0866 ms/query。IVF-HNSW 对 nprobe 更敏感: nprobe=16 时延迟较低但 recall 不足, nprobe=32 达到阈值后延迟已到 0.2488 ms/query, nprobe=64 进一步增加到 0.4266 ms/query。该结果说明图索引组合的优化变量并不独立, ef 扩大单图搜索宽度, nprobe 增加被搜索的小图数量, 两者相乘后会放大常数开销。

ARM quick 中, HNSW 与 multi-HNSW raw recall 接近 Flat 上限, 但 latency 分别为 0.2412 和 0.2942 ms/query。IVF-HNSW 的最低延迟为 0.2071 ms/query, 但 raw recall 只有 0.4606; 若追求更高 raw recall, 需要 nprobe=64, latency 会升至约 0.6461 ms/query。这个结果说明 IVF-HNSW 是典型的负优化候选: 结构更复杂, 不一定降低延迟; 只有当簇内数据足够大、簇选择足够准确, 或多个簇内 HNSW 能并行搜索时, 它才可能优于直接 HNSW。

图索引构建也需要纳入 trade-off。HNSW 构建需要逐点插入并维护邻接关系, x86 P1 构建约 6.42 s, P2 降到约 2.77 s, P4 降到约 1.31 s。IVF 构建更快、参数更直观, 适合频繁重建或快速调参; HNSW

查询延迟更低, 适合离线构建、在线查询的场景。multi-HNSW 适合追求更高候选覆盖, 但要接受 merge 成本上升。

11 综合分析

11.1 串行、MPI、MPI+SIMD、MPI+OpenMP 的关系

Flat scalar 单进程是最直接的串行基线; Flat SIMD 单进程体现距离核优化; 多 rank Flat/IVF 体现 MPI 数据分片; OpenMP T2/T4 体现进程内 query-level 并行; HNSW 和 IVF-HNSW 体现算法层面的访问量削减。实验结果表明, 性能提升并非来自单一技术: MPI 减少每 rank 数据量, SIMD 降低每次距离计算成本, OpenMP 提高 query batch 并发度, ANN 索引减少访问点数。四者叠加后, 瓶颈会从距离计算转移到 top-k 堆、通信和 merge。

11.2 通信与计算重叠

当前 nonblocking 版本没有把通信与下一批搜索重叠, 因此优势不稳定。要真正重叠, 需要将 query batch 拆成多个块, rank 在搜索 batch b 时异步发送 batch $b-1$ 的候选, root 分批 merge。这会把总时间从

$$T = \sum_b (T_{\text{search},b} + T_{\text{comm},b} + T_{\text{merge},b})$$

改为近似

$$T \approx T_{\text{search},0} + \sum_b \max(T_{\text{search},b}, T_{\text{comm},b-1}) + \sum_b T_{\text{merge},b}.$$

但该方案要求双缓冲、结果顺序管理和更复杂的错误处理。在当前规模下, 通信占比只有在 ARM quick P4 和更高 ranks 时较明显, 因此优先级低于减少 merge 成本。

11.3 merge/reduce 优化

当前 root 直接合并所有 rank 的候选, 简单且稳定, 但不是最优通信结构。对于多节点集群, 可以使用树形 top-k reduce: 相邻 rank 先局部合并, 再逐层向 root 汇总。该方法把 root 处理的瞬时候选压力从 $P \times k$ 分散到多层 $2k$ 合并, 降低 root 热点, 但实现需要自定义 top-k 数据类型或手写分层通信。另一种策略是每 rank 返回 top-p, 且 $p < 100$, 减少通信和 merge; 其风险是召回下降, 必须重新进行 recall-latency 消融。

11.4 局部 top-p 与 rerank

当前实现让每个 rank 固定返回 top-100。这一选择保证全局 merge 的语义清晰: 任意真实全局 top-100 如果落入某个 rank, 本地搜索只要命中该点, 就一定会进入该 rank 的返回集。若改为 top-p 且 $p < 100$, 通信量会从 $P \times 100$ 降到 $P \times p$, root merge 的候选数也同步下降; 但当某个 rank 包含大量真实近邻时, 过小的 p 会直接截断这些候选, 导致 recall 下降。由于最终目标是 recall@100 不低于阈值后的最低 latency, top-p 应作为单独消融参数, 而不能只按通信量最小选择。

rerank 方面, 本文的 Flat、IVF、HNSW 和组合索引在候选进入堆时已经使用原始 float 向量计算精确内积, 因此没有额外的近似编码重排阶段。若后续迁移到 IVF-PQ 或压缩图索引, 搜索阶段可以先用量化距离产生更大的候选池, 再用 SIMD 精确距离对候选做 rerank。该设计会把开销从大范围扫描转移到小候选集上的精确重排, 适合与 MPI top-p 返回一起分析: 每个 rank 返回更多未重排候

选可能提高 recall，但会增加通信；每个 rank 先本地 rerank 则减少通信，但需要更多本地精确距离计算。

阶段分析的重点是解释瓶颈迁移，而不是只比较最终 latency。x86 IVF 的 Pareto 前沿定位 nprobe=32，阶段拆分说明 P4 以后 merge 占比上升，通信方式对比则说明四种通信 API 的差异小于算法参数差异。由此可以看出，后续优化应优先减少 root 合并压力和无效候选传输，再考虑把 nonblocking 通信改造成真正的 batch 流水线。

11.5 自由探索：候选返回与分层合并

除基础 base-sharding 外，本文额外分析了候选返回规模、树形合并和 batch 流水线三个方向。固定返回 top-100 的好处是语义稳定，不会在 MPI 回收前截断真实近邻；缺点是通信量和 root merge 随 ranks 线性增长。top-p 返回可以减少候选量，但必须重新做 recall-latency 消融，因为某个 rank 可能包含大量真实近邻。树形 top-k reduce 可以把 root 热点分散到多层合并，但需要自定义候选规约逻辑。batch 流水线则尝试让搜索与通信重叠，但需要双缓冲和顺序控制。实验中通信 API 的直接替换收益有限，因此这些自由探索更适合作为下一步结构性优化，而不是当前最快配置。

11.6 平台差异

x86 full 使用 AVX2 和较高单核频率，IVF/HNSW 查询延迟显著低于 ARM quick 和 ARM full qsub 的同类路径；ARM NEON 在 Flat 上加速比更高，说明 ARM 标量路径更慢且向量化收益明显。ARM quick 中通信占比更高，既与 reduced-base 导致本地搜索缩短有关，也可能与 MPI 运行时和内存系统有关。ARM full qsub 则给出了同规模判断：HNSW P2T4 的 latency 为 0.1318 ms/query，是 x86 同配置 0.0330 ms/query 的约 4.0 倍，但 recall 均约为 0.981。跨平台比较不能只看绝对 latency，应同时看问题规模、raw recall 上限、stage breakdown、SIMD speedup 和作业运行方式。

更具体地说，x86 full 的 100k base/1000 query 让本地搜索占比更高，通信和 root merge 被较长的 search 阶段摊薄；ARM quick 的 50k base/300 query 缩短了本地搜索，固定通信同步和进程启动波动更容易在 per-query latency 中显现。ARM full qsub 消除了 reduced-base 的 recall 上限问题，但保留了 Kunpeng 平台的频率、内存层次、MPI 实现和 PBS 运行环境差异。SIMD 方面，AVX2 一次处理 8 个 float，NEON 一次处理 4 个 float，但实际加速比还受标量基线、编译器调度和内存带宽影响。Flat 中距离核占比最高，所以 NEON 加速比可以很高；IVF/HNSW 中堆维护、倒排链跳转和图访问占比提高，指令宽度差异就不再直接等价于端到端速度差异。

11.7 优化过程复盘

整体优化可以分为六个阶段。第一阶段实现 Flat base-sharding，用精确 recall 验证数据切分、全局 id、候选回收和 root merge。这个阶段的关键不是追求最低延迟，而是确保 P1/P2/P4/P8 返回结果与单机精确语义一致；Flat P8 recall=1.0 说明通信与 merge 没有破坏 top-100 排序。

第二阶段加入 SIMD 距离核。Flat 标量 P1 为 3.0393 ms/query，Flat SIMD P1 降到 1.7449 ms/query；P8 下标量为 0.4937 ms/query，SIMD 为 0.1928 ms/query。这个结果说明距离计算仍是扫描型方法的主瓶颈，也验证了 kernel 分发在 MPI 程序中没有引入额外语义差异。

第三阶段引入 IVF，把优化目标从“更快地扫描全部向量”转向“减少候选访问量”。nprobe=16 的延迟最低但 recall 不足；nprobe=32 在 P8 达到 0.9654 recall 和 0.0573 ms/query，是 IVF 路径的最优点；nprobe=64/128/256 的 recall 更高，但延迟随链扫描量增长。该阶段确立了后续所有实验的筛选逻辑：先过滤掉 recall 不达标的点，再在剩余点中选 latency 最低。

第四阶段加入 OpenMP 和 `std::thread`。IVF 中 OpenMP 能进一步压缩 batch query 的搜索时间，但当 ranks 很多时收益变小；HNSW 中 P1T4 最快，说明图索引访问量已经足够低，继续拆分 base 反而增加 merge。这个阶段把优化重点从“增加并行度”转为“匹配 ranks 与 threads 的比例”，最终得到 HNSW P2T4 作为 MPI 多进程检查配置。

第五阶段探索通信方式、MPI_THREAD_MULTIPLE、multi-HNSW 和 IVF-HNSW。结果显示通信 API 的差异小于算法参数差异，MULTIPLE 在主线程通信结构下没有明显收益，组合图索引能提高 recall 但不一定降低延迟。因此最终代码保留这些分支作为实验和分析工具，而不是把它们设为默认最快路径。

第六阶段在 ARM/Kunpeng 上先用 quick sweep 检查趋势，再把 HNSW 候选放回 full-data qsub 验证。quick 结果用于判断 NEON、OpenMP、ranks 和通信方式的相对行为，full-data 结果用于最终阈值判断。验证表明 P2T4 在 ARM 上仍是更合理的 MPI 多进程配置；P4T2 和 P8T1 虽然提高 recall，但 latency 受多分片图搜索、候选回收和 merge 影响明显上升。

11.8 最终选择

x86 full 的最终选择按 $\text{recall}@100 \geq 0.95$ 和 latency 最小确定。若允许单 rank + OpenMP，HNSW P1T4 是最低延迟点；若强调 MPI 多进程参与，HNSW P2T4 是最低延迟点；若选择倒排索引路径，IVF P8 nprobe=32 是最优点。Flat 只作为精确基线；IVF-HNSW 和 multi-HNSW 提供了图索引组合实验和 trade-off 分析，但不是最低延迟方案。

最终 MPI 多进程配置为 HNSW、nonblocking、OpenMP、SIMD、2 ranks、4 threads、100k base、1000 query、ef=64、repeat=3。该配置不是 x86 全表最低的 P1T4，而是在保留 MPI 多进程参与的前提下满足 recall 阈值并取得最低 latency；在 ARM full qsub 上，该配置同样是已验证候选中的最低延迟点。完整消融解释了为什么更多 ranks、更高 ef 或更复杂的组合图索引没有成为最终选择。

12 结论

本文完成了 MPI base-sharding ANN 查询框架，实现了本地索引构建、局部 top-100 维护、多种通信回收方式和全局 top-100 merge，并将 SIMD 与 OpenMP 混合并行纳入同一实验矩阵。x86 full 中，HNSW P1T4 达到 0.0312 ms/query、 $\text{recall}@100=0.9545$ ；MPI 多进程条件下 HNSW P2T4 达到 0.0330 ms/query、 $\text{recall}@100=0.9816$ ；IVF P8 nprobe=32 达到 0.0573 ms/query、 $\text{recall}@100=0.9654$ 。ARM quick 结果显示同一代码在 AArch64/NEON 上呈现与 x86 一致的 nprobe、ranks、OpenMP 和 SIMD 趋势；ARM full qsub 进一步验证 HNSW P2T4 在 100k/1000 数据上达到 0.1318 ms/query、 $\text{recall}@100=0.9814$ ，因此 ARM 平台也有满足阈值的最终候选。

实验的主要结论是：MPI 分片对扫描型检索和 IVF 有稳定收益；SIMD 仍是距离计算核心优化；OpenMP 在每 rank 工作量充足时有效；通信方式差异小于算法和 nprobe 差异；当本地搜索足够快时，merge 和通信会成为扩展瓶颈。双平台结果说明 P2T4 这类“少量 MPI 分片 + rank 内批并行”的配置比盲目增加 ranks 更稳定。后续可围绕 query batch 流水线、树形 top-k reduce、top-p 自适应返回、全局 centroid 同步和图索引分片剪枝继续优化。

参考文献

- [1] Y. A. Malkov and D. A. Yashunin. Efficient and robust approximate nearest neighbor search using Hierarchical Navigable Small World graphs. *IEEE TPAMI*, 2020.
- [2] W. Fu, C. Xiang, Y. Wang, and D. Cai. Fast Approximate Nearest Neighbor Search With The Navigating Spreading-out Graph. *PVLDB*, 2019.
- [3] Message Passing Interface Forum. MPI: A Message-Passing Interface Standard.
- [4] A. Andre, A.-M. Kermarrec, and N. Le Scouarnec. Cache locality is not enough: high-performance nearest neighbor search with product quantization fast scan. *VLDB*, 2015.

A 生成式 AI 辅助学习记录

问答一：为什么 ANN 的 MPI 并行更常使用 base-sharding？

用户问题：ANN 检索中，MPI 进程应该切分 query 还是切分 base data？

AI 回答：两种切分方式对应不同系统目标。query-sharding 把不同 query 分配给不同进程，每个进程独立处理一部分请求，通信量很低；但为了得到全局 top-k，每个进程必须拥有完整 base 或完整索引。这样会复制内存，并且进程数增加时不能扩大可管理数据规模。base-sharding 则把 base data 分布在不同进程，每个进程只保存一部分向量或局部索引。query 广播给所有进程后，每个进程返回局部 top-k，最后合并成全局 top-k。它的通信量更高，但可以真正分摊数据和计算，结果也与完整数据集的 ground truth 对齐。对于 DEEP100K 这类固定 top-100 评测，base-sharding 更适合分析 MPI 数据并行、负载均衡、通信开销和 merge/reduce 开销。

采纳与影响：实现采用 base-sharding 作为主线，报告也围绕局部分片搜索、局部 top-100 和全局 merge 展开。query-sharding 只作为复杂性对照，用于说明为什么不能简单把 query 分给多个进程后复制完整索引。

问答二：IVF 的 nprobe 如何影响 MPI 版本性能？

用户问题：为什么 IVF 的 nprobe 是最重要的消融参数？

AI 回答：nprobe 决定每个 query 访问多少倒排簇。nprobe 小时，访问候选少，延迟低，但真实近邻可能落在未访问簇中；nprobe 大时，覆盖更多候选，召回提高，但扫描量增加。MPI 分片后，每个 rank 的倒排链更短，因此同一 nprobe 下本地扫描变快；同时每个 rank 都返回 top-k，root 的候选池扩大到 $P \times k$ 。这会带来一个折中：增加 ranks 降低本地 search，但提高通信和 merge 压力；增加 nprobe 提高 recall，但提高本地扫描。实验中 x86 full 的 IVF 最优点是 P8 nprobe=32，说明它刚好跨过 recall@100=0.95 阈值，而更大的 nprobe 带来的召回收益不值得其额外延迟。

采纳与影响：IVF 实验把 nprobe 设为核心消融变量，并与 ranks 一起画出 recall-latency 曲线。最终选择 P8、nprobe=32，是基于阈值约束下的延迟最小，而不是选择 recall 更高但扫描量更大的 nprobe=64/128/256。

问答三：nonblocking MPI 为什么没有自动带来加速？

用户问题：非阻塞通信不是可以重叠通信和计算吗，为什么结果中优势不明显？

AI 回答：非阻塞通信提供的是“发起后可继续执行”的能力，而不是自动重叠。若程序结构是先搜索全部 query，再统一 `MPI_Igather`，那么通信阶段前面没有可并行的计算，`MPI_Igather` 仍然需要在 `MPI_Waitall` 处等待完成。要发挥 nonblocking 的优势，需要把 query 分成多个 batch：搜索 batch b 时传输 batch $b - 1$ 的候选，root 同时合并更早 batch。这要求双缓冲、异步请求管理和结果顺序控制。当前实现以可比性和稳定计时为优先，因此 nonblocking 主要作为通信方法对比，而不是完整流水线优化。

采纳与影响：通信实验把 blocking、nonblocking、p2p 和 one-sided 放在相同结果格式下比较，并在正文中把 nonblocking 的实际收益与流水线条件分开讨论。该结论影响了优化顺序：先分析 merge 和候选返回规模，再考虑 batch 流水线。

问答四：MPI_THREAD_MULTIPLE 适合什么场景？

用户问题：MPI 支持多线程后，是否应该总是请求 MPI_THREAD_MULTIPLE？

AI 回答：不应该。若只有主线程调用 MPI，MPI_THREAD_FUNNELED 已经足够。请求 MPI_THREAD_MULTIPLE 会要求 MPI 运行时保证多个线程同时进入 MPI 的安全性，内部可能需要锁或更复杂的进度管理。只有当程序确实让多个线程同时发起 `MPI_Isend`、`MPI_Irecv`、`MPI_Put` 等操作时，MULTIPLE 才可能有价值。对于本文的 query-level OpenMP，本地搜索线程不调用 MPI，所以 MULTIPLE 与 FUNNELED 差异很小。若后续实现 batch 流水线通信，可以重新评估 MULTIPLE 是否帮助隐藏通信延迟。

采纳与影响：主程序默认使用 FUNNELED，并补充 MULTIPLE 对比实验。正文将该实验解释为 MPI 线程安全支持的开销测量，而不是把 MULTIPLE 当作自动优化开关。

问答五：为什么图索引在 MPI 中容易出现负优化？

用户问题：HNSW 本身很快，为什么 multi-HNSW 或 IVF-HNSW 不一定更快？

AI 回答：HNSW 快是因为图导航能访问少量候选点。但当它与 MPI 分片组合时，每个 rank 都会执行一次本地图搜索，并返回 top-k；root 合并的候选数随 ranks 增加。multi-HNSW 提高了候选覆盖，但也增加了并行图搜索和合并成本。IVF-HNSW 还多了 centroid 选择和多个簇内 HNSW 查询，小图搜索的固定开销会被重复支付。如果 nprobe 小，召回不足；如果 nprobe 大，多个簇内图搜索的总时间可能高于直接 HNSW。因此图索引的并行化不能只叠加结构，需要分析每层过滤是否真正减少了足够多的访问点。

采纳与影响：图索引实验同时保留 HNSW、multi-HNSW 和 IVF-HNSW。报告没有把组合结构默认为更优，而是用 latency、recall 和构建时间解释 multi-HNSW 提高覆盖、IVF-HNSW 增加常数开销的 trade-off。

问答六：SIMD 与 MPI 的收益为什么会相互影响？

用户问题：距离核已经用 SIMD 加速，为什么 MPI ranks 增加后 SIMD 加速比可能下降？

AI 回答：SIMD 只加速距离计算部分。MPI ranks 增加后，每个 rank 的本地扫描量下降，距离计算在总时间中的占比也下降；与此同时，通信、merge、top-k 堆维护等不可向量化部分占比上升。根据 Amdahl 定律，当不可加速部分比例变大时，SIMD 的总体可见加速比会下降。实验中 IVF 在 x86 P1 的 SIMD 加速比约 $3.10\times$ ，P8 下降到约 $2.09\times$ ，正是因为 MPI 分片后搜索时间被压缩，merge 和通信占比提高。

采纳与影响： SIMD 实验按平台和算法族分别计算加速比，并结合 stage breakdown 解释加速比变化。该分析避免只报告 AVX2/NEON 的局部内核速度，而忽略 MPI 通信和 merge 对端到端延迟的影响。